

Plano de Negócio para Solopreneur

<https://gemini.google.com/app/4b41a21cbc0d0291>

User prompt: Sei gerir Sei desenvolver Dou suporte técnico Tenho conhecimento dos nichos e segmentos de mercado Tenho formacao em ti Tenho tempo disponível Tenho uma conta gemini pro Nao tenho grana pra aplicar no empreendimento Não tenho pessoas pra montar equipe... Digo isso para que me ajude e mostre como fazer isso se tranformar em um negocio de prestação de serviços ou venda de produto/software/app. Se precisar de mais informação pra tracar meu perfil e ter insiths melhores, me diga. Torne essa converda interativa ate chegarmos num plano final.

Response: Você tem exatamente o conjunto de habilidades que o ecossistema de **solopreneur em tecnologia (empresa de uma pessoa só)** exige: capacidade técnica de ponta a ponta (desenvolvimento + suporte) e visão de gestão de produto/negócio.

O caminho ideal para quem tem **tempo e conhecimento técnico, mas caixa zero e sem equipe**, não é tentar construir um SaaS gigante do zero (o que consome meses sem retorno), mas sim o modelo de **transição orientada a caixa**:

- Fase 1 (Serviço de alto valor / Implantação rápida):** Gera receita imediata, valida dores reais de empresas e cria fluxo de caixa.
- Fase 2 (Productized Service):** Você empacota uma solução recorrente com escopo fechado e suporte contínuo.
- Fase 3 (Micro-SaaS / Software próprio):** Você automatiza e transforma o problema recorrente que resolveu manualmente em um produto vendável, já financiado pelo caixa da Fase 1 e 2.

Para desenharmos um plano cirúrgico e viável para você começar a operar em dias, me responda estas 4 perguntas:

- Qual é a sua stack de desenvolvimento mais confortável?** (Ex.: Python, Node.js, PHP, React, automações no-code/low-code, APIs etc.)
- Entre os nichos de mercado que você mais conhece, quais 2 ou 3 você domina profundamente os processos internos?** (Ex.: academias/fitness, escritórios jurídicos, logística, clínicas, e-commerce, consultorias, serviços corporativos?)
- Qual modelo te atrai mais de imediato?**
 - A) Automatizar e integrar processos para PMEs usando IA/APIs (ciclo de venda rápido, cobrança de setup + mensalidade).
 - B) Consultoria e suporte técnico especializado como braço direito de pequenas empresas.
 - C) Construção direta de um micro-software/MVP voltado a uma dor muito específica de um nicho que você conhece bem.
- Quantas horas por semana você consegue dedicar a isso agora?**

Mande essas respostas e vamos estruturar a oferta, o público-alvo inicial e a estratégia de aquisição sem gastar um real com anúncios.

User prompt: 1. Uso python, angular, flutter, java, etc 2. Nessa ordem: academias/fitness, clinicas, serviço corporativos 3. Mvp 4. Entre 40h e 50h

Response: Com **Python** no backend, **Flutter/Angular** no front, **40 a 50 horas semanais** dedicadas e domínio do nicho de **academias/fitness e clínicas**, você tem a combinação exata para validar e colocar um MVP funcional no ar em menos de 3 semanas gastando praticamente zero.

No nicho fitness/saúde, os grandes softwares de gestão (ERP de catraca, mensalidade e agendamento) já estão saturados. Onde está a brecha lucrativa para um MVP ágil de uma pessoa só? **Na ponta técnica do atendimento, retenção de alunos e prescrição/acompanhamento.**

Aqui estão 3 ideias de MVP viáveis para você construir e vender sozinho:

Opção 1: Agente IA de Prescrição & Adaptação de Treinos (B2B para Personais e Consultorias)

- A dor:** Personais e donos de estúdios perdem horas montando planilhas e respondendo dúvidas de alunos no WhatsApp sobre substituição de exercícios, ajustes de carga ou feedback de execução.
- O MVP:** Um WebApp (Angular + FastAPI/Python) ou bot integrado ao WhatsApp com Gemini Pro. O personal cadastra a anamnese e diretrizes do aluno; a IA gera o split inicial e responde ao aluno no WhatsApp com substituições inteligentes de exercícios caso um aparelho esteja ocupado ou haja desconforto articular, respeitando os limites pré-definidos pelo treinador.
- Modelo de receita:** R\$ 79 a R\$ 199/mês por personal trainer (assinatura recorrente).

Opção 2: Sistema de Avaliação Física com Visão Computacional / Postural no Celular

- A dor:** Avaliações posturais e biofotogrametria tradicionais exigem softwares caros, computadores dedicados ou régua fixas na parede, tornando o processo lento para academias e clínicas de fisioterapia.
- O MVP:** App mobile em Flutter que captura fotos/vídeos dos planos frontal e sagital, usa modelos leves em Python para detectar pontos anatômicos e gera na hora um relatório visual em PDF com simetria, desvios e evolução temporal.
- Modelo de receita:** Cobrança por pacote de avaliações (ex.: R\$ 2 por relatório) ou plano mensal ilimitado para clínicas/estúdios (R\$ 150 a R\$ 300/mês).

Opção 3: Recuperador Inteligente de Alunos Ausentes (Retenção via WhatsApp)

- A dor:** A maior perda financeira de academias e estúdios de médio/pequeno porte é a evasão ("churn"). O aluno para de ir na 3ª semana e ninguém percebe até o plano vencer.

- **O MVP:** Script/integração em Python que lê a frequência (seja via planilha exportada do sistema atual da academia ou integração simples) e dispara fluxos de reativação humanizados e personalizados via IA no WhatsApp antes do cancelamento acontecer.
- **Modelo de receita:** Setup de implantação (R\$ 500) + mensalidade de manutenção (R\$ 250 a R\$ 400/mês).

Para escolhermos a direção exata e montarmos o cronograma de desenvolvimento dos próximos 14 dias:

1. Qual dessas três vertentes faz mais sentido para você:
- A) Foco em **treinadores/pessoais** (ganho de escala e suporte automatizado).
 - B) Foco em **biomecânica/avaliação postural** (tecnologia visual e valor perceptível alto).
 - C) Foco em **faturamento da academia** (recuperação de receita e retenção).

2. Você prefere construir primeiro como WebApp (Angular) ou App Mobile (Flutter)?

Me diga qual te empolga mais e já fechamos o escopo do MVP, a infraestrutura gratuita/freemium para rodar e o roteiro de abordagem comercial direta.

User prompt: 1. Opção 1: Agente IA de Prescrição & Adaptação de Treinos (B2B para Pessoais e Consultorias) 2. Flutter

Response: A escolha da **Opção 1 com Flutter** é cirúrgica: pessoais e consultorias online vivem no celular (WhatsApp e Instagram). Ter uma interface mobile nativa, fluida e prática para o treinador gerenciar seus alunos e para o aluno interagir é um diferencial enorme frente a sistemas web legados e burocráticos.

Como o objetivo é **validar rápido, gastar R\$ 0,00 e operar sozinho**, o escopo precisa ser focado no que resolve a dor central sem dispersão técnica.

1. Escopo Enxuto do MVP (O que construir)

O produto conecta duas pontas no mesmo aplicativo Flutter (com perfis de Treinador e Aluno) consumindo um backend leve em Python:

- **Perfil Treinador:**
 - 1. Cadastro rápido de aluno (dados básicos, limitações articulares/lesões, objetivo, local de treino: academia comercial, condomínio ou em casa).
 - 2. Gerador assistido de ficha (o Gemini Pro monta a periodização inicial baseada nos parâmetros do treinador, e o treinador aprova ou edita em 1 clique).
 - 3. Dashboard simples de suporte: vê o histórico de dúvidas e adaptações solicitadas pelos alunos.
- **Perfil Aluno:**
 - 1. Visualização do treino do dia (exercício, séries, repetições, descanso).
 - 2. Botão de emergência "**Aparelho Ocupado / Dor**": a IA sugere instantaneamente uma substituição biomecanicamente equivalente com base nos equipamentos disponíveis e nas restrições cadastradas.
 - 3. Chat tira-dúvidas de execução focado estritamente no treino prescrito.

2. Arquitetura Técnica Custo Zero (Free Tier)

Para rodar sem gastar nada no início:

Camada	Tecnologia	Hospedagem / Custo
App Mobile	Flutter (Android primeiro via APK/Closed Track)	Custo R\$ 0
Backend API	Python (FastAPI)	Render, Fly.io ou Railway (Free Tier)
Banco de Dados	PostgreSQL + Auth	Supabase (Free Tier com 500 MB e Auth embutido)
Inteligência	Gemini API (Gemini Pro)	Google AI Studio (Free Tier generoso para testes e primeiros clientes)

3. Cronograma de Execução (14 Dias / Dedicação 40h-50h/sem)

Semana 1: Backend e Mecânica de IA

- **Dias 1 e 2:** Modelagem no Supabase (tabelas: users, profiles, anamnesis, workouts, exercises, adaptations_log). Setup do FastAPI com autenticação via JWT/Supabase.
- **Dias 3 a 5:** Integração com o Gemini Pro via Python. Engenharia de prompt restritiva (System Prompt estruturado com saída em JSON para geração de treinos e lógica de substituição biomecânica: mesmo padrão de movimento, sinergistas e restrições).
- **Dias 6 e 7:** Testes de carga e refinamento dos prompts para garantir respostas rápidas e sem alucinações técnicas.

Semana 2: App Flutter e Validação

- **Dias 8 a 10:** Telas essenciais no Flutter: Login, Lista de Alunos, Editor de Ficha (Treinador) e Tela de Treino Ativo + Botão de Adaptação (Aluno).
- **Dias 11 e 12:** Integração Flutter com a API FastAPI, validação de fluxo de ponta a ponta e build de teste (APK).
- **Dias 13 e 14:** Gravação de vídeo demo (1 minuto mostrando o personal gerando a ficha e o aluno substituindo um exercício na academia) e início da prospecção direta.

4. Próximo Passo: O Prompt Central do Motor de IA

Para começarmos a estruturar o backend, o coração do sistema é o **System Prompt da engine em Python**.

Ele precisa receber a anamnese e cuspir um JSON estrito para o Flutter renderizar direto, sem quebrar o layout.

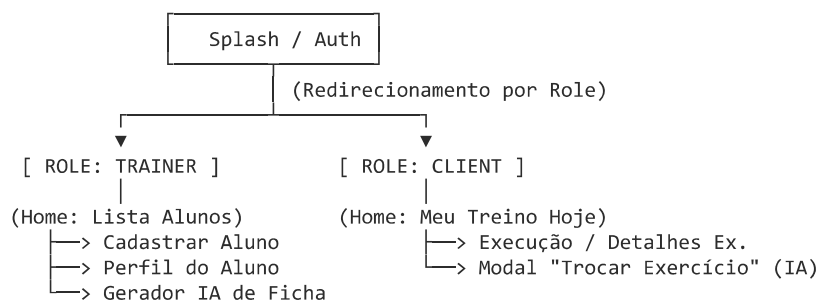
Quer que eu desenhe a **especificação do JSON do treino e o prompt de substituição de exercícios**, ou prefere definir primeiro o **fluxo de navegação das telas no Flutter**?

User prompt: Primeiro o fluxo de navegação

Response: Para o MVP ser rápido de codificar em Flutter e intuitivo para o usuário, a regra de ouro é: **poucas telas, navegação rasa e zero menus escondidos**.

O app terá um ponto de entrada único (Login/Auth via Supabase), que redireciona imediatamente pelo campo role (trainer ou client).

1. Diagrama de Navegação



2. Fluxo do Treinador (Foco em Escalar Atendimento)

O objetivo do treinador é cadastrar um aluno em menos de 2 minutos e aprovar uma ficha gerada por IA com poucos toques.

- **Tela T1: Home (Lista de Alunos)**
 - Listagem com foto/nome, status (Ativo/Inativo) e badge de alerta se o aluno solicitou alguma adaptação recente.
 - Botão de ação flutuante (FAB): + Novo Aluno.
- **Tela T2: Cadastro Rápido / Anamnese**
 - Formulário direto com campos essenciais: Objetivo (Hipertrofia, Emagrecimento, etc.), Nível (Iniciante, Intermediário, Avançado), Dias por semana, Local (Academia completa, Condomínio, Peso corporal) e **Restrições/Lesões** (ex.: "Dor no ombro direito - evitar abdução acima de 90°").
 - Botão: Gerar Ficha com IA.
- **Tela T3: Validador da Ficha Gerada (Review & Edit)**
 - Recebe o JSON estruturado do backend e exibe a divisão em abas (Treino A, B, C...).
 - Cada exercício exibe nome, séries, reps e descanso.
 - O personal pode editar manualmente qualquer valor, reordenar ou clicar em **Aprovar** e **Liberar** para o Aluno.
- **Tela T4: Histórico de Adaptações do Aluno**
 - Acessível tocando no aluno na Home. Mostra o que a IA alterou em tempo real para o aluno na academia (ex.: "Trocou Leg Press por Agachamento Goblet devido a aparelho ocupado"), mantendo o treinador 100% no controle.

3. Fluxo do Aluno (Foco em Agilidade no Salão de Treino)

O aluno não quer preencher formulário enquanto treina. A interface precisa de botões grandes, contraste alto e resposta imediata.

- **Tela A1: Home (Treino do Dia)**
 - Cabeçalho simples: Nome da ficha do dia (ex.: *Treino A - Peito e Tríceps*).
 - Lista de cards verticais dos exercícios em ordem de execução.
 - Cada card tem: Imagem/GIF ilustrativo curto, Séries x Reps, Carga sugerida e um botão de alerta: Trocar Aparelho.

- **Modal A2: Diálogo Rápido de Adaptação (Gatilho da IA)**
 - Ao tocar em Trocar Aparelho, abre um *bottom sheet* com 2 opções de 1 clique:
 1. "Aparelho Ocupado / Fila"
 2. "Desconforto ou Dor"
 - O app consulta o backend via Gemini e substitui o card na hora por uma variação com o mesmo vetor biomecânico e compatível com as restrições cadastradas.
- **Tela A3: Detalhe do Exercício (Opcional/Inline)**
 - Dicas de biomecânica e instrução curta de postura para evitar erros comuns de execução.

4. Gerenciamento de Estado no Flutter

Para construir isso de forma limpa e sem boilerplate excessivo em uma semana:

- **Gerenciador de Estado: Provider** ou **Riverpod** (ou até `flutter_bloc` se você já tiver templates prontos). Se quiser velocidade máxima de codificação, **Signals** ou **Riverpod** reduzem muito o código repetitivo.
- **Rotas:** `go_router` com `redirect` declarativo baseado no estado da sessão do Supabase:
 - Sessão nula → `/login`
 - Sessão ativa + `role == 'trainer'` → `/trainer/home`
 - Sessão ativa + `role == 'client'` → `/client/home`

Esse fluxo atende perfeitamente ao que você precisa para o MVP sem excesso de telas.

Podemos avançar para o **contrato de dados da API e o System Prompt (FastAPI + Gemini)** que vai alimentar esse fluxo de geração e substituição de treinos?

User prompt: Sim

Response: Aqui está a espinha dorsal técnica do seu backend: o **contrato JSON estrito** que o Flutter vai consumir e o **System Prompt do Gemini Pro** dentro de uma rota FastAPI.

A regra fundamental aqui é usar o recurso nativo de **Structured Outputs (JSON Schema)** da API do Gemini, garantindo que o modelo nunca devolva texto solto ou markdown conversacional, apenas o payload que o Flutter consegue desserializar direto.

1. Contrato de Dados (JSON Schema)

O payload retornado pela IA para a **Tela T3 (Revisão da Ficha)** deve seguir exatamente este formato:

```
{
  "workout_plan_title": "Hipertrofia ABC - Foco Peito e Membros Inferiores",
  "notes_for_trainer": "Volume moderado respeitando a queixa de desconforto anterior no ombro direito.",
  "splits": [
    {
      "split_identifier": "A",
      "split_name": "Peito, Ombros e Tríceps",
      "estimated_duration_min": 50,
      "exercises": [
        {
          "order": 1,
          "name": "Supino Reto com Halteres",
          "target_muscle_group": "Peitoral Maior",
          "sets": 4,
          "reps": "8-10",
          "rest_seconds": 90,
          "notes": "Manter escápulas retraídas e cotovelos a ~45-60 graus do tronco.",
          "substitution_vector": "Empurrar horizontal livre"
        }
      ]
    }
  ]
}
```

E para a **adaptação imediata na academia (Modal A2)**:

```
{
  "original_exercise": "Supino Reto com Halteres",
  "adapted_exercise": "Supino na Máquina Articulada",
  "reason": "Redução do torque instável no ombro direito mantendo o mesmo padrão de movimento e volume equivalente.",
  "sets": 4,
  "reps": "10-12",
  "rest_seconds": 60,
```

```

    "notes": "Ajustar o banco para que os pegadores fiquem na linha média do peitoral."
}

```

2. Implementação no FastAPI com Gemini Pro

Instale as dependências mínimas:

```
pip install fastapi uvicorn google-genai pydantic
```

Aqui está a implementação enxuta do endpoint em Python (main.py), pronta para rodar em containers ou free tiers (como Fly.io/Render):

```

import os
from typing import List, Optional
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from google import genai
from google.genai import types

app = FastAPI(title="Trainer AI Engine")

# Inicializa o cliente do Gemini usando a nova SDK oficial
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

# --- Modelos Pydantic (Entrada) ---
class AnamnesisInput(BaseModel):
    objective: str # ex: "Hipertrofia", "Emagrecimento"
    training_level: str # "Iniciante", "Intermediário", "Avançado"
    days_per_week: int # 3 a 6
    workout_location: str # "Academia completa", "Condomínio", "Em casa"
    injuries_or_restrictions: Optional[str] = "Nenhuma restrição articular relatada."

class AdaptationInput(BaseModel):
    current_exercise: str
    reason: str # "Aparelho Ocupado" ou "Dor / Desconforto"
    workout_location: str
    injuries_or_restrictions: Optional[str] = "Nenhuma"

# --- Modelos Pydantic (Saída Estruturada) ---
class Exercise(BaseModel):
    order: int
    name: str
    target_muscle_group: str
    sets: int
    reps: str
    rest_seconds: int
    notes: str
    substitution_vector: str

class Split(BaseModel):
    split_identifier: str
    split_name: str
    estimated_duration_min: int
    exercises: List[Exercise]

class WorkoutPlanResponse(BaseModel):
    workout_plan_title: str
    notes_for_trainer: str
    splits: List[Split]

class AdaptationResponse(BaseModel):
    original_exercise: str
    adapted_exercise: str
    reason: str
    sets: int
    reps: str
    rest_seconds: int
    notes: str

# --- Endpoints da API ---

@app.post("/api/v1/generate-plan", response_model=WorkoutPlanResponse)
async def generate_plan(data: AnamnesisInput):
    system_instruction = """
    Você é um especialista em biomecânica e prescrição de treinamento de força de alto nível.
    Sua missão é gerar divisões de treino balanceadas, seguras e orientadas a resultados.
    Diretrizes obrigatórias:
    1. Respeite com rigor absoluto quaisquer lesões ou dores relatadas; NUNCA prescreva movimentos com contraindicações.
    2. Adequar a seleção de exercícios ao local (ex.: se 'Condomínio', não assuma máquinas sofisticadas).
    3. Manter o volume compatível com o nível do praticante (iniciante: 10-12 séries semanais por grupamento; interm
    4. Gere a resposta estritamente no schema solicitado, sem preâmbulos ou texto conversacional.

```

```

"""

user_prompt = f"""
Gerar periodização com os dados:
- Objetivo: {data.objective}
- Nível: {data.training_level}
- Frequência: {data.days_per_week} dias/semana
- Local: {data.workout_location}
- Restrições/Dores: {data.injuries_or_restrictions}
"""

try:
    response = client.models.generate_content(
        model="gemini-2.5-pro",
        contents=user_prompt,
        config=types.GenerateContentConfig(
            system_instruction=system_instruction,
            response_mime_type="application/json",
            response_schema=WorkoutPlanResponse,
            temperature=0.2, # Baixa criatividade para garantir precisão e estabilidade biomecânica
        ),
    )
    return WorkoutPlanResponse.model_validate_json(response.text)
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@app.post("/api/v1/adapt-exercise", response_model=AdaptationResponse)
async def adapt_exercise(data: AdaptationInput):
    system_instruction = """
    Você é um consultor biomecânico em tempo real de apoio a alunos de musculação.
    Quando um exercício precisa ser trocado na academia:
    1. Se o motivo for 'Aparelho Ocupado': Encontre uma variação imediata com o MESMO padrão motor e estímulo muscul
    2. Se o motivo for 'Dor / Desconforto': Substitua por um movimento biomecanicamente mais amigável, reduzindo o t
    3. Respeite sempre as restrições cadastradas.
    4. Responda estritamente de acordo com o schema JSON configurado.
    """

    user_prompt = f"""
    Substituir o exercício:
    - Exercício atual: {data.current_exercise}
    - Motivo: {data.reason}
    - Ambiente/Equipamentos: {data.workout_location}
    - Histórico de lesões: {data.injuries_or_restrictions}
    """

    try:
        response = client.models.generate_content(
            model="gemini-2.5-pro",
            contents=user_prompt,
            config=types.GenerateContentConfig(
                system_instruction=system_instruction,
                response_mime_type="application/json",
                response_schema=AdaptationResponse,
                temperature=0.1,
            ),
        )
        return AdaptationResponse.model_validate_json(response.text)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

3. Modelagem no Supabase (Camada de Persistência Gratuita)

Para conectar o Flutter e a API sem complicação, você precisará apenas de 4 tabelas simples no Supabase:

1. **profiles:** id (chave estrangeira de auth.users), role ('trainer' ou 'client'), name, trainer_id (para associar o aluno ao personal).
2. **anamnesis:** id, client_id, objective, level, restrictions, location.
3. **workouts:** id, client_id, trainer_id, plan_title, plan_json (onde você armazena o JSON completo gerado e aprovado), is_active (boolean).
4. **adaptation_logs:** id, client_id, original_exercise, adapted_exercise, reason, created_at (permite que o personal audite no Flutter o que foi alterado).

Próximo Passo

Com o backend desenhado e o fluxo estabelecido, temos duas opções para o próximo bloco de trabalho:

1. **Implementação do cliente no Flutter:** Criar os modelos em Dart (freezed ou json_serializable) e a camada de serviço que chama essa API FastAPI e gerencia o estado da ficha.
2. **Validação e Prospecção:** Montar o roteiro de abordagem direta (script de WhatsApp / Direct) para conseguir seus primeiros 3 a 5 pessoais de teste antes mesmo de terminar o polimento visual.

Por qual dos dois quer seguir agora?

User prompt: Siga pelo fluxo lógico

Response: Seguindo a esteira lógica de produto, o próximo passo antes de ir para o mercado é **estruturar a camada cliente no Flutter**: conectar os modelos de dados em Dart com a API FastAPI, implementar a camada de serviço e montar a tela-chave do MVP (**Tela A1/Modal A2 - Treino do Dia com Substituição em Tempo Real**).

Essa tela é o "uau" do produto: o momento em que o aluno aperta um botão e o app resolve o problema sem intervenção manual imediata do treinador.

1. Modelos Dart (lib/models/workout_models.dart)

Para mapear diretamente o JSON da API sem complexidade de bibliotecas de geração de código neste momento, usamos modelos Dart puros com métodos fromJson:

```
class ExerciseModel {
  final int order;
  final String name;
  final String targetMuscleGroup;
  final int sets;
  final String reps;
  final int restSeconds;
  final String notes;
  final String substitutionVector;

  ExerciseModel({
    required this.order,
    required this.name,
    required this.targetMuscleGroup,
    required this.sets,
    required this.reps,
    required this.restSeconds,
    required this.notes,
    required this.substitutionVector,
  });

  factory ExerciseModel.fromJson(Map<String, dynamic> json) {
    return ExerciseModel(
      order: json['order'] as int,
      name: json['name'] as String,
      targetMuscleGroup: json['target_muscle_group'] as String,
      sets: json['sets'] as int,
      reps: json['reps'] as String,
      restSeconds: json['rest_seconds'] as int,
      notes: json['notes'] as String,
      substitutionVector: json['substitution_vector'] as String,
    );
  }

  ExerciseModel copyWith({
    String? name,
    int? sets,
    String? reps,
    int? restSeconds,
    String? notes,
  }) {
    return ExerciseModel(
      order: order,
      name: name ?? this.name,
      targetMuscleGroup: targetMuscleGroup,
      sets: sets ?? this.sets,
      reps: reps ?? this.reps,
      restSeconds: restSeconds ?? this.restSeconds,
      notes: notes ?? this.notes,
      substitutionVector: substitutionVector,
    );
  }
}

class AdaptationResult {
  final String originalExercise;
```

```

final String adaptedExercise;
final String reason;
final int sets;
final String reps;
final int restSeconds;
final String notes;

AdaptationResult({
  required this.originalExercise,
  required this.adaptedExercise,
  required this.reason,
  required this.sets,
  required this.reps,
  required this.restSeconds,
  required this.notes,
});

factory AdaptationResult.fromJson(Map<String, dynamic> json) {
  return AdaptationResult(
    originalExercise: json['original_exercise'] as String,
    adaptedExercise: json['adapted_exercise'] as String,
    reason: json['reason'] as String,
    sets: json['sets'] as int,
    reps: json['reps'] as String,
    restSeconds: json['rest_seconds'] as int,
    notes: json['notes'] as String,
  );
}

```

2. Serviço de Integração (lib/services/trainer_api_service.dart)

Camada responsável por disparar a requisição de substituição para a FastAPI:

```

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/workout_models.dart';

class TrainerApiService {
  // Ajuste para o endereço da sua instância FastAPI (Render, Fly ou local)
  static const String baseUrl = 'http://10.0.2.2:8000/api/v1';

  static Future<AdaptationResult> requestAdaptation({
    required String currentExercise,
    required String reason,
    required String workoutLocation,
    String? restrictions,
  }) async {
    final url = Uri.parse('$baseUrl/adapt-exercise');

    final response = await http.post(
      url,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'current_exercise': currentExercise,
        'reason': reason,
        'workout_location': workoutLocation,
        'injuries_or_restrictions': restrictions ?? 'Nenhuma',
      })),
    );

    if (response.statusCode == 200) {
      final data = jsonDecode(utf8.decode(response.bodyBytes));
      return AdaptationResult.fromJson(data);
    } else {
      throw Exception('Falha ao processar adaptação: ${response.statusCode}');
    }
  }
}

```

3. Tela de Treino do Aluno com Substituição em Tempo Real (lib/views/client_workout_view.dart)

Esta interface entrega a usabilidade necessária para o aluno no salão da academia: cartões limpos com destaque para séries, repetições, tempo de descanso e o acionador do bottom sheet de troca.

```

import 'package:flutter/material.dart';
import '../models/workout_models.dart';
import '../services/trainer_api_service.dart';

```

```

class ClientWorkoutView extends StatefulWidget {
  const ClientWorkoutView({super.key});

  @override
  State<ClientWorkoutView> createState() => _ClientWorkoutViewState();
}

class _ClientWorkoutViewState extends State<ClientWorkoutView> {
  // Exemplo de estado local com exercícios do dia
  List<ExerciseModel> exercises = [
    ExerciseModel(
      order: 1,
      name: "Supino Reto com Halteres",
      targetMuscleGroup: "Peitoral Maior",
      sets: 4,
      reps: "8-10",
      restSeconds: 90,
      notes: "Manter escápulas aduzidas e punhos firmes.",
      substitutionVector: "Empurrar horizontal livre",
    ),
    ExerciseModel(
      order: 2,
      name: "Desenvolvimento com Halteres",
      targetMuscleGroup: "Deltoide Anterior/Lateral",
      sets: 3,
      reps: "10-12",
      restSeconds: 60,
      notes: "Controle na descida até a linha das orelhas.",
      substitutionVector: "Empurrar vertical livre",
    ),
  ];

  bool isLoading = false;

  void _triggerAdaptationModal(int exerciseIndex) {
    final currentExercise = exercises[exerciseIndex];

    showModalBottomSheet(
      context: context,
      shape: const RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
      ),
      builder: (ctx) {
        return Padding(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                'Adaptar: ${currentExercise.name}',
                style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 8),
              const Text(
                'Por qual motivo você precisa substituir este exercício agora?',
                style: TextStyle(color: Colors.grey),
              ),
              const SizedBox(height: 20),
              ElevatedButton.icon(
                icon: const Icon(Icons.people_outline),
                label: const Text('Aparelho Ocupado / Fila'),
                style: ElevatedButton.styleFrom(padding: const EdgeInsets.all(14)),
                onPressed: () {
                  Navigator.pop(ctx);
                  _executeAdaptation(exerciseIndex, 'Aparelho Ocupado');
                },
              ),
              const SizedBox(height: 12),
              OutlinedButton.icon(
                icon: const Icon(Icons.healing_outlined, color: Colors.orange),
                label: const Text('Desconforto ou Dor Articular', style: TextStyle(color: Colors.orange)),
                style: OutlinedButton.styleFrom(padding: const EdgeInsets.all(14)),
                onPressed: () {
                  Navigator.pop(ctx);
                  _executeAdaptation(exerciseIndex, 'Dor / Desconforto');
                },
              ),
              const SizedBox(height: 12),
            ],
          ),
        );
      },
    );
  }
}

```

```

    ],
  ),
);
},
);
}

Future<void> _executeAdaptation(int index, String reason) async {
  setState(() => isLoading = true);

  try {
    final current = exercises[index];
    final result = await TrainerApiService.requestAdaptation(
      currentExercise: current.name,
      reason: reason,
      workoutLocation: 'Academia completa',
      restrictions: 'Leve desconforto em ombro direito',
    );

    setState(() {
      exercises[index] = current.copyWith(
        name: result.adaptedExercise,
        sets: result.sets,
        reps: result.reps,
        restSeconds: result.restSeconds,
        notes: '${result.notes} (Motivo: ${result.reason})',
      );
    });

    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          backgroundColor: Colors.green.shade800,
          content: Text('Substituído com sucesso por: ${result.adaptedExercise}'),
        ),
      );
    }
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          backgroundColor: Colors.red.shade800,
          content: Text('Erro ao adaptar exercício: $e'),
        ),
      );
    }
  } finally {
    setState(() => isLoading = false);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Treino A - Peito e Ombros'),
    ),
    body: Stack(
      children: [
        ListView.separated(
          padding: const EdgeInsets.all(16),
          itemCount: exercises.length,
          separatorBuilder: (_, __) => const SizedBox(height: 14),
          itemBuilder: (context, index) {
            final item = exercises[index];
            return Card(
              elevation: 2,
              shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
              child: Padding(
                padding: const EdgeInsets.all(16.0),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      mainAxisAlignment: MainAxisAlignment.spaceBetween,
                      children: [
                        Expanded(
                          child: Text(
                            '${item.order}. ${item.name}',

```

```

                style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
            ),
        ),
        IconButton(
            icon: const Icon(Icons.swap_horiz, color: Colors.blueAccent),
            tooltip: 'Trocar exercício',
            onPressed: () => _triggerAdaptationModal(index),
        ),
    ],
),
Text(item.targetMuscleGroup, style: const TextStyle(color: Colors.grey, fontSize: 13)),
const Divider(height: 24),
Row(
    mainAxisAlignment: MainAxisAlignment.spaceAround,
    children: [
        _MetricBadge(label: 'Séries', value: '${item.sets}'),
        _MetricBadge(label: 'Reps', value: item.reps),
        _MetricBadge(label: 'Descanso', value: '${item.restSeconds}s'),
    ],
),
if (item.notes.isNotEmpty) ...[
    const SizedBox(height: 12),
    Container(
        padding: const EdgeInsets.all(8),
        decoration: BoxDecoration(
            color: Colors.grey.shade100,
            borderRadius: BorderRadius.circular(6),
        ),
        child: Text(
            item.notes,
            style: TextStyle(fontSize: 12, color: Colors.grey.shade800),
        ),
    ),
],
],
),
),
);
},
),
if (isLoading)
    Container(
        color: Colors.black38,
        child: const Center(
            child: CircularProgressIndicator(),
        ),
    ),
],
),
);
}
}

class _MetricBadge extends StatelessWidget {
  final String label;
  final String value;

  const _MetricBadge({required this.label, required this.value});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(label, style: const TextStyle(fontSize: 11, color: Colors.grey)),
        const SizedBox(height: 2),
        Text(value, style: const TextStyle(fontSize: 15, fontWeight: FontWeight.bold)),
      ],
    );
  }
}

```

O Que Faltava para Fechar o Ciclo do MVP

Com a API pronta e a tela principal do aluno estruturada, o fluxo lógico aponta para as **duas últimas peças do quebra-cabeça**:

1. **A Tela T2/T3 do Treinador (Front-end):** O formulário de anamnese rápida que chama /generate-plan e renderiza a ficha completa para revisão antes de salvar no Supabase.
2. **Estratégia de Validação com Usuários Reais:** O script direto para abordar 5 a 10 pessoais nas suas redes/academias locais para instalarem o APK e testarem gratuitamente com 2 alunos, coletando feedbacks críticos.

Qual dessas duas etapas construímos agora?

User prompt: 1

Response: Aqui está a implementação da **Tela do Treinador** no Flutter, unificando a **Anamnese Rápida (T2)** e o **Validador de Ficha Gerada por IA (T3)** com suporte a edição manual e persistência.

A tela utiliza um fluxo de duas etapas (PageView ou alternância de estado) para manter a experiência fluida: o treinador preenche 5 campos essenciais, clica em "Gerar Periodização com IA", confere a ficha dividida em abas (Treinos A, B, C...) e pode editar exercícios antes de salvar.

1. Modelos Completos em Dart (lib/models/plan_models.dart)

Adicione o modelo que mapeia o payload completo retornado por /generate-plan:

```
import 'workout_models.dart';

class SplitModel {
  final String splitIdentifier;
  final String splitName;
  final int estimatedDurationMin;
  List<ExerciseModel> exercises;

  SplitModel({
    required this.splitIdentifier,
    required this.splitName,
    required this.estimatedDurationMin,
    required this.exercises,
  });

  factory SplitModel.fromJson(Map<String, dynamic> json) {
    return SplitModel(
      splitIdentifier: json['split_identifier'] as String,
      splitName: json['split_name'] as String,
      estimatedDurationMin: json['estimated_duration_min'] as int,
      exercises: (json['exercises'] as List)
        .map((e) => ExerciseModel.fromJson(e as Map<String, dynamic>))
        .toList(),
    );
  }
}

class WorkoutPlanModel {
  final String title;
  final String notesForTrainer;
  final List<SplitModel> splits;

  WorkoutPlanModel({
    required this.title,
    required this.notesForTrainer,
    required this.splits,
  });

  factory WorkoutPlanModel.fromJson(Map<String, dynamic> json) {
    return WorkoutPlanModel(
      title: json['workout_plan_title'] as String,
      notesForTrainer: json['notes_for_trainer'] as String,
      splits: (json['splits'] as List)
        .map((s) => SplitModel.fromJson(s as Map<String, dynamic>))
        .toList(),
    );
  }
}
```

2. Método de Chamada na Camada de Serviço (TrainerApiService)

Adicione o método de geração no arquivo lib/services/trainer_api_service.dart:

```
static Future<WorkoutPlanModel> generateWorkoutPlan({
  required String objective,
  required String trainingLevel,
  required int daysPerWeek,
  required String workoutLocation,
  required String restrictions,
}) async {
  final url = Uri.parse('$baseUrl/generate-plan');

  final response = await http.post(
    url,
```

```

headers: {'Content-Type': 'application/json'},
body: jsonEncode({
  'objective': objective,
  'training_level': trainingLevel,
  'days_per_week': daysPerWeek,
  'workout_location': workoutLocation,
  'injuries_or_restrictions': restrictions.isEmpty ? 'Nenhuma' : restrictions,
})),
);

if (response.statusCode == 200) {
  final data = jsonDecode(utf8.decode(response.bodyBytes));
  return WorkoutPlanModel.fromJson(data);
} else {
  throw Exception('Erro ao gerar periodização: ${response.statusCode}');
}
}

```

3. Tela do Treinador: Anamnese e Revisor (lib/views/trainer_builder_view.dart)

```

import 'package:flutter/material.dart';
import '../models/plan_models.dart';
import '../models/workout_models.dart';
import '../services/trainer_api_service.dart';

class TrainerBuilderView extends StatefulWidget {
  const TrainerBuilderView({super.key});

  @override
  State<TrainerBuilderView> createState() => _TrainerBuilderViewState();
}

class _TrainerBuilderViewState extends State<TrainerBuilderView> {
  final _formKey = GlobalKey<FormState>();

  // Controladores do Formulário de Anamnese
  String objective = 'Hipertrofia Muscular';
  String level = 'Intermediário';
  int daysPerWeek = 4;
  String location = 'Academia completa';
  final TextEditingController restrictionsCtrl = TextEditingController(
    text: 'Desconforto anterior em ombro direito (evitar abdução excessiva)',
  );

  bool isLoading = false;
  WorkoutPlanModel? generatedPlan;

  Future<void> _handleGenerate() async {
    if (!_formKey.currentState!.validate()) return;
    setState(() => isLoading = true);

    try {
      final plan = await TrainerApiService.generateWorkoutPlan(
        objective: objective,
        trainingLevel: level,
        daysPerWeek: daysPerWeek,
        workoutLocation: location,
        restrictions: restrictionsCtrl.text,
      );

      setState(() {
        generatedPlan = plan;
      });
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(backgroundColor: Colors.red, content: Text('Erro: $e')),
      );
    } finally {
      setState(() => isLoading = false);
    }
  }

  void _editExerciseDialog(SplitModel split, int exIndex) {
    final ex = split.exercises[exIndex];
    final setsCtrl = TextEditingController(text: ex.sets.toString());
    final repsCtrl = TextEditingController(text: ex.reps.toString());

    showDialog(
      context: context,

```

```

builder: (ctx) => AlertDialog(
  title: Text('Editar: ${ex.name}'),
  content: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      TextField(
        controller: setsCtrl,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(labelText: 'Séries'),
      ),
      TextField(
        controller: repsCtrl,
        decoration: const InputDecoration(labelText: 'Repetições (ex: 8-10)'),
      ),
    ],
  ),
  actions: [
    TextButton(onPressed: () => Navigator.pop(ctx), child: const Text('Cancelar')),
    ElevatedButton(
      onPressed: () {
        setState(() {
          split.exercises[exIndex] = ex.copyWith(
            sets: int.tryParse(setsCtrl.text) ?? ex.sets,
            reps: repsCtrl.text,
          );
        });
        Navigator.pop(ctx);
      },
      child: const Text('Salvar'),
    ),
  ],
),
);
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(generatedPlan == null ? 'Nova Ficha (Assistente IA)' : 'Revisar Ficha'),
      actions: [
        if (generatedPlan != null)
          IconButton(
            icon: const Icon(Icons.refresh),
            tooltip: 'Recomeçar',
            onPressed: () => setState(() => generatedPlan = null),
          )
      ],
    ),
    body: Stack(
      children: [
        generatedPlan == null ? _buildAnamnesisForm() : _buildPlanReviewer(),
        if (isLoading)
          Container(
            color: Colors.black45,
            child: const Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.min,
                children: [
                  CircularProgressIndicator(),
                  SizedBox(height: 16),
                  Text(
                    'Calculando volume e biomecânica com Gemini Pro...',
                    style: TextStyle(color: Colors.white, fontWeight: FontWeight.w500),
                  ),
                ],
              ),
            ),
          ),
      ],
    ),
  );
}

Widget _buildAnamnesisForm() {
  return Form(
    key: _formKey,
    child: ListView(
      padding: const EdgeInsets.all(20),

```

```

children: [
  const Text(
    'Parâmetros do Aluno',
    style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
  ),
  const SizedBox(height: 16),
  DropdownButtonFormField<String>(
    value: objective,
    decoration: const InputDecoration(labelText: 'Objetivo', border: OutlineInputBorder()),
    items: ['Hipertrofia Muscular', 'Emagrecimento', 'Condicionamento Geral']
      .map((e) => DropdownMenuItem(value: e, child: Text(e)))
      .toList(),
    onChanged: (v) => setState(() => objective = v!),
  ),
  const SizedBox(height: 16),
  DropdownButtonFormField<String>(
    value: level,
    decoration: const InputDecoration(labelText: 'Nível de Treino', border: OutlineInputBorder()),
    items: ['Iniciante', 'Intermediário', 'Avançado']
      .map((e) => DropdownMenuItem(value: e, child: Text(e)))
      .toList(),
    onChanged: (v) => setState(() => level = v!),
  ),
  const SizedBox(height: 16),
  DropdownButtonFormField<int>(
    value: daysPerWeek,
    decoration: const InputDecoration(labelText: 'Dias por Semana', border: OutlineInputBorder()),
    items: [3, 4, 5, 6].map((e) => DropdownMenuItem(value: e, child: Text('$e dias'))).toList(),
    onChanged: (v) => setState(() => daysPerWeek = v!),
  ),
  const SizedBox(height: 16),
  DropdownButtonFormField<String>(
    value: location,
    decoration: const InputDecoration(labelText: 'Ambiente de Treino', border: OutlineInputBorder()),
    items: ['Academia completa', 'Condomínio', 'Em casa (Halteres/Peso Corporal)']
      .map((e) => DropdownMenuItem(value: e, child: Text(e)))
      .toList(),
    onChanged: (v) => setState(() => location = v!),
  ),
  const SizedBox(height: 16),
  TextFormField(
    controller: restrictionsCtrl,
    maxLines: 3,
    decoration: const InputDecoration(
      labelText: 'Dores, Lesões ou Restrições Articulares',
      border: OutlineInputBorder(),
      alignLabelWithHint: true,
    ),
  ),
  const SizedBox(height: 24),
  ElevatedButton.icon(
    icon: const Icon(Icons.auto_awesome),
    label: const Text('Gerar Ficha Personalizada com IA'),
    style: ElevatedButton.styleFrom(
      padding: const EdgeInsets.symmetric(vertical: 16),
      textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
    ),
    onPressed: _handleGenerate,
  ),
],
),
);
}

```

```

Widget _buildPlanReviewer() {
  final plan = generatedPlan!;
  return DefaultTabController(
    length: plan.splits.length,
    child: Column(
      children: [
        Container(
          color: Theme.of(context).primaryColor.withOpacity(0.08),
          padding: const EdgeInsets.all(12),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(plan.title, style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 16)),
              const SizedBox(height: 4),
              Text(

```

```

        'Diretriz da IA: ${plan.notesForTrainer}',
        style: TextStyle(fontSize: 12, color: Colors.grey.shade700),
      ),
    ],
  ),
),
TabBar(
  isScrollable: true,
  labelColor: Theme.of(context).primaryColor,
  tabs: plan.splits
    .map((s) => Tab(text: 'Treino ${s.splitIdentifier} (${s.splitName})'))
    .toList(),
),
Expanded(
  child: TabBarView(
    children: plan.splits.map((split) {
      return ListView.builder(
        padding: const EdgeInsets.all(12),
        itemCount: split.exercises.length,
        itemBuilder: (ctx, i) {
          final ex = split.exercises[i];
          return Card(
            margin: const EdgeInsets.only(bottom: 10),
            child: ListTile(
              leading: CircleAvatar(child: Text('${ex.order}')),
              title: Text(ex.name, style: const TextStyle(fontWeight: FontWeight.bold)),
              subtitle: Text(
                '${ex.sets} séries x ${ex.reps} reps | Descanso: ${ex.restSeconds}s\n${ex.notes}',
                style: const TextStyle(fontSize: 12),
              ),
              trailing: IconButton(
                icon: const Icon(Icons.edit, size: 20),
                onPressed: () => _editExerciseDialog(split, i),
              ),
            ),
          );
        },
      );
    }).toList(),
  ),
),
Padding(
  padding: const EdgeInsets.all(16.0),
  child: SizedBox(
    width: double.infinity,
    child: ElevatedButton.icon(
      icon: const Icon(Icons.check_circle_outline),
      label: const Text('Aprovar e Liberar para o Aluno'),
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.green.shade700,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.all(16),
      ),
      onPressed: () {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Ficha aprovada e sincronizada com sucesso!')),
        );
      },
    ),
  ),
),
),
),
),
),
),
),
);
}
}

```

Com o app tendo a **visão do Treinador (prescrição em 30 segundos com controle de edições)** e a **visão do Aluno (execução limpa e botão anti-fila/dor em tempo real)**, o software está funcionalmente fechado para validação.

O próximo passo lógico para destravar receita é a **Estratégia de Aquisição e Validação**: como empacotar esse app e abordar os primeiros 5 pessoais para testar e fechar as primeiras assinaturas. Quer avançar para essa etapa?

User prompt: Sim

Response: Para validar o MVP sem gastar um centavo em tráfego pago, a estratégia consiste em **venda consultiva e validação assistida** diretamente com pessoais e consultorias online locais ou da sua rede.

O personal trainer não quer "mais um software de academia" para perder tempo cadastrando coisas. O que vende de imediato é resolver duas dores que tomam o tempo dele:

1. **O tempo perdido montando planilhas do zero no domingo à noite.**
2. **As mensagens no WhatsApp no meio do dia com o aluno perguntando:** *"O leg press tá lotado, o que eu faço?"* ou *"Meu joelho incomodou nesse, troco pelo quê?"*.

Aqui está o plano tático de 3 etapas para fechar seus primeiros **3 a 5 clientes pagantes** nos próximos 10 dias.

Etapa 1: A Demonstração Magnética (Vídeo de 45 segundos)

Gere um APK de testes (ou grave a tela do emulador). O vídeo não precisa de edição profissional, apenas gravação de tela com áudio limpo:

- **00:00 a 00:15:** Mostre você preenchendo a anamnese no app com uma restrição real (*"Dor no ombro direito"*), tocando em **Gerar Ficha** e o Gemini Pro entregando a periodização dividida em menos de 5 segundos.
- **00:16 a 00:30:** Mude para a visão do aluno. Toque em **Trocar Aparelho** → escolha *"Aparelho Ocupado"* → a IA substitui na hora por um halter livre mantendo o mesmo estímulo biomecânico.
- **00:31 a 00:45:** Finalize dizendo: *"Você foca em vender consultorias e prescrever com rigor; o app cuida do suporte operacional do aluno no salão da academia."*

Etapa 2: O Roteiro de Abordagem Direta (WhatsApp / Direct do Instagram)

Aborde de 15 a 20 pessoais que trabalham com consultoria online ou atendem alunos em academias de bairro/redes. Não tente vender de primeira; convide para um **acesso pioneiro de validação**.

Mensagem Direta:

"Fala, [Nome], tudo bem?"

Acompanho seu trabalho aqui com consultoria e sei que o tempo gasto ajustando ficha ou respondendo aluno no WhatsApp sobre aparelho ocupado no meio do treino consome boa parte do dia.

Eu desenvolvi um motor mobile para pessoais que usa IA para gerar a estrutura da periodização respeitando as lesões do aluno em segundos (para você só revisar e aprovar) e, melhor ainda: quando o aluno tá treinando e o aparelho tá lotado, o próprio app dá uma substituição biomecânica equivalente aprovada pelas suas diretrizes, sem ele precisar te parar.

Gravei um vídeo de 40 segundos mostrando como funciona: [Link do Vídeo ou Vídeo Anexo].

Estou liberando acesso gratuito para 5 treinadores rodarem com 2 ou 3 alunos essa semana para coletar feedback de usabilidade. Topa dar uma olhada e testar com um aluno seu sem custo?"

Etapa 3: Oferta Irresistível de Fechamento (De Beta Gratuito para Assinante)

Após o personal rodar o app por 5 a 7 dias com 2 alunos, você agenda uma conversa de 10 minutos (ou troca áudios rápidos) para pegar o feedback:

1. *"O que seus alunos acharam da troca de exercício no salão?"*
2. *"Quanto tempo você economizou na montagem da ficha?"*

Com as respostas positivas em mãos, você faz a proposta de **Membro Fundador**:

- **O Modelo:** Em vez de cobrar R\$ 150/mês logo de início, você oferece uma condição vitalícia:
 - *"Estou finalizando os últimos ajustes antes de abrir publicamente. Para os 5 primeiros pessoais que me ajudaram a validar, fecho o plano ilimitado (quantos alunos quiser) por **R\$ 69/mês**, fixo para sempre enquanto você mantiver a assinatura."*
- **A Matemática do Início:**
 - $5 \text{ pessoais} \times \text{R\$ } 69/\text{mês} = \text{R\$ } 345/\text{mês de MRR (Receita Recorrente)}$.
 - O custo da infraestrutura (FastAPI no Fly/Render + Supabase + API do Gemini) para esse volume gira em torno de **R\$ 0 a R\$ 20/mês**.
 - Margem líquida quase total para você reinvestir ou pagar as contas enquanto escala.

Checklist dos Próximos 3 Dias

1. [] **Dia 1:** Conectar o Flutter com o backend FastAPI local ou publicado no Render/Fly.io.
2. [] **Dia 2:** Gerar o APK de release (`flutter build apk --release`) e gravar o vídeo demo de 45 segundos.
3. [] **Dia 3:** Enviar a mensagem para 15 pessoais e colocar o app na mão dos primeiros 3 treinadores.

User prompt: *So nao entendi como sera a assinatura, quando chegar o momento de fechar esse acordo*

Response: Para quem está começando sozinho e sem caixa, **o maior erro é tentar programar um gateway de pagamento complexo dentro do app antes de validar a venda**. Isso consome dias de código inútil.

Nos primeiros 5 a 10 clientes, a cobrança deve ser o mais simples possível, sem intermediários que cobrem taxas de adesão.

O processo de cobrança se divide em duas etapas:

Fase 1: Validação Inicial (Primeiros 5 clientes) → Manual e Pix Automático

Aqui o foco é atrito zero e custo zero:

1. **Plataforma de Cobrança:** Use plataformas brasileiras com link de pagamento e recorrência pronta, como **Asaas**, **Mercado Pago** ou **OpenPix**.
 - O **Asaas** é excelente para isso: você cria um link de assinatura recorrente (via Pix ou Cartão de Crédito) em 2 minutos pela web, sem programar uma linha.
2. **O Fechamento na Prática:**
 - Quando o personal disser "gostei, quero continuar usando", você envia o link direto pelo WhatsApp:
"Show, [Nome]! Para travar sua condição de Membro Fundador por R\$ 69/mês, é só ativar por este link seguro: [link-da-assinatura]. Assim que confirmar, sua conta já fica com acesso liberado e ilimitado."
3. **Liberação no Banco:**
 - Você vai no Supabase e simplesmente muda o status da conta dele de status: 'trial' para status: 'active'.

Fase 2: Automatizada (A partir do 6º cliente) → Webhook Direto no Supabase

Assim que você validar que as pessoas realmente pagam, você automatiza a liberação:

1. **Webhook:** Configura um endpoint simples no seu FastAPI (/api/v1/webhook/payment).
2. **Fluxo:**
 - O cliente paga o Pix ou Cartão pelo link do Asaas/Stripe/Mercado Pago.
 - O gateway dispara um POST para a sua API avisando que a fatura foi paga.
 - Sua API atualiza o registro do personal no Supabase: `is_subscribed = true` e `subscription_valid_until = data_futura`.
3. **Controle no App:**
 - Quando o personal abre o Flutter, o app checa essa flag. Se estiver ativa, libera tudo; se vencer, mostra um aviso amigável com o botão para renovar.

Por que não usar compra in-app da Google Play / Apple Store agora?

- As lojas cobram de 15% a 30% de taxa sobre cada mensalidade.
- A aprovação de assinaturas nas lojas exige burocracias fiscais e políticas de cancelamento detalhadas que atrasam o lançamento em semanas.
- Como o personal trainer é um **prestador de serviços B2B**, vender a assinatura dele via web/Pix direto pelo WhatsApp é 100% comum no Brasil e mantém 100% do dinheiro no seu bolso.

User prompt: Liste aqui a solução que delineamos e seus passos a seguir. Vamos iniciar!

Response: Aqui está o plano consolidado de ponta a ponta da nossa solução, estruturado para você começar a executar imediatamente.

Visão Geral da Solução

- **Produto:** Plataforma mobile B2B para Personal Trainers e Consultorias Fitness.
- **Proposta de Valor Central:**
 1. *Para o Treinador:* Gera divisões de treino completas e biomecanicamente seguras em segundos via IA, prontas para revisão e aprovação rápida.
 2. *Para o Aluno:* No salão da academia, resolve o problema de aparelho ocupado ou desconforto articular em 1 clique, gerando substituição imediata e compatível com as orientações do treinador.
- **Stack Tecnológica (Custo Zero Inicial):**
 - **App:** Flutter (Android APK direto).
 - **Backend:** FastAPI (Python) com Structured Outputs (JSON Schema).
 - **Motor de IA:** Gemini Pro API.
 - **Banco & Auth:** Supabase (PostgreSQL Free Tier).
 - **Hospedagem API:** Render ou Fly.io (Free Tier).

- **Modelo de Negócio Inicial:** Assinatura B2B recorrente (R\$ 69/mês para os primeiros clientes / "Membros Fundadores") cobrada via Link de Pagamento/Pix (Asaas ou Mercado Pago), sem taxas de app store.

Roteiro de Execução Passo a Passo

[Fase 1: Fundação] → [Fase 2: Conexão & Build] → [Fase 3: Demo & Validação] → [Fase 4: Cobrança]
(Banco + API) (Flutter + APK) (Vídeo + Prospecção) (Primeiros R\$)

Fase 1: Backend e Banco de Dados (Dias 1 a 3)

1. Configurar Supabase:

- Criar projeto gratuito.
- Criar tabelas essenciais: profiles (com flag role e subscription_status), workouts, anamnesis e adaptation_logs.

2. Subir o Backend FastAPI:

- Implementar os endpoints /api/v1/generate-plan e /api/v1/adapt-exercise.
- Configurar a chave da API do Gemini (GEMINI_API_KEY) e validar os schemas de resposta em JSON estrito.
- Fazer deploy da API no Render ou Fly.io (ou rodar local via ngrok para testes imediatos).

Fase 2: Aplicativo Flutter (Dias 4 a 7)

1. Fluxo do Aluno:

- Tela de treino do dia listando exercícios, séries e repetições.
- Modal/Bottom Sheet com as opções "Aparelho Ocupado" e "Dor Articular", disparando a requisição para o backend e atualizando o card na tela.

2. Fluxo do Treinador:

- Formulário de anamnese rápida (objetivo, nível, dias, local, restrições).
- Tela de revisão com abas (Treinos A, B, C...) permitindo editar séries/repetições e aprovar a ficha.

3. Build:

- Gerar o instalador standalone do Android: flutter build apk --release.

Fase 3: Demonstração e Abordagem Direta (Dias 8 a 10)

1. Gravar Vídeo Demo de 45 Segundos:

- 15s: Preenchendo a anamnese e a IA gerando a ficha completa.
- 15s: Visão do aluno tocando em "Trocar Aparelho" e a substituição aparecendo na hora.
- 15s: Chamada direta destacando o ganho de tempo e escala para a consultoria do personal.

2. Prospecção de 15 Treinadores:

- Disparo do script de abordagem no WhatsApp/Instagram convidando 5 pessoais para testarem gratuitamente por 7 dias com 2 alunos.

Fase 4: Conversão e Primeiras Assinaturas (Dias 11 a 14)

1. Coleta de Feedback:

- Contato rápido para saber quanto tempo o treinador economizou e o retorno dos alunos.

2. Fechamento "Membro Fundador":

- Oferta do plano vitalício a R\$ 69/mês.
- Envio de link de pagamento (Asaas/Pix recorrente) e ativação manual do status no Supabase.

O pontapé inicial é criar as tabelas no Supabase ou inicializar o projeto FastAPI localmente. Por onde você quer abrir os trabalhos agora?
